

Why These Models?

A plain-language walkthrough of how PhishGuard picks its brain.

PhishGuard's job: look at a URL and say whether it's a phishing link - WITHOUT visiting the website. It only reads the address itself (length, weird subdomains, '@' tricks, entropy, and so on). That means the model has to learn from 18 simple numbers per URL. Two models do this job in this project: LogisticRegression and RandomForest. Here's why each one is there.

The two models, in one sentence each

- **LogisticRegression (LR):** adds up weighted clues and squashes the total into a 0-100% phishing score. Simple, fast, and you can read exactly why it decided anything.
- **RandomForest (RF):** asks 200 simpler 'decision trees' to vote. Each tree sees a random slice of the data; the majority wins. Stronger, but a tangle of thousands of rules.

What we measured (21,471 held-out real URLs)

Model	F1	ROC-AUC	Speed (per URL)
RandomForest	0.888	0.954	slow (~20 s load)
LogisticRegression	0.804	0.885	instant (~0.3 s)

RF wins on paper. So why does the website run LR? Because the two products have different jobs - and that distinction drives everything below.

Decision 1 - the website runs LogisticRegression

The site must run inside your browser, with no server. That means the model has to be exported as plain data. The two models are wildly different in how easy that is:

- **LR is just arithmetic:** a list of 18 weights, one bias, and the formula $\text{score} = \text{sigmoid}(\text{bias} + \sum(\text{weight} \times \text{feature}))$. That fits in a ~2 KB JSON file, and a browser reproduces Python's answers EXACTLY - same math, digit for digit.
- **RF is a library:** 200 trees with thousands of branching rules can't be translated to the browser without either rebuilding all of sklearn in JavaScript or shipping a 53 MB blob. Not practical.

And the twist that makes it a fair trade: the small accuracy gap mostly reflects harder examples, not a broken model. LR still catches what URLs look like when they're fishy - and the site shows you exactly which features pushed its verdict, with real numbers.

The trade in one line:

We gave up ~8 points of F1 to gain exact reproducibility, instant startup, and per-verdict explanations in the browser.

Decision 2 - RandomForest stays as the accuracy benchmark

If LR were hopelessly weak, the trade above would be dishonest. RF answers that question. It is trained and evaluated on exactly the same data and features, and its higher F1 (0.888 vs 0.804) proves the 18 features carry real signal that a stronger learner can exploit. It also gives the Results page its two most useful pictures: the confusion matrix and the feature-importance ranking (hostname length, path length, entropy...). In short: LR is the product, RF is the ceiling.

Decision 3 - why not XGBoost, a neural net, or k-NN?

- **Gradient boosting (XGBoost/LightGBM):** usually the top scorer on tables like this - but it adds a heavy dependency and would ALSO be stuck in Python with the same export problem as RF. All cost, no product benefit over RF.
- **Neural network:** shines on raw text, images, or audio. Here the inputs are already 18 tidy numbers; a neural net would be a bigger, opaque version of LR with the same job. Overkill.
- **k-NN / Naive Bayes / SVM:** kept as honest baselines during development - but each loses to LR+RF on some axis that matters (speed, exportability, or accuracy), so none earned a place.

Decision 4 - train on 107,355 real URLs, not toy data

An early run on a 244-URL toy dataset scored a suspicious F1 of 1.000 - the classes were trivially separable by top-level domain. The fix was the dataset, not the model:

- **Phishing:** 48,338 verified URLs from the PhishTank feed.
- **Legitimate:** Tranco top-10k domains PLUS real deep links, so both classes contain URLs with paths - otherwise the model learns 'has a path = phishing', which is dataset bias, not knowledge.
- **Hygiene:** dedupe everything, and cap any single host at 10 URLs so the model learns transferable patterns instead of memorizing hosts.

Decision 5 - recall over precision

The two errors are not equal: flagging a good URL is annoying; missing a phishing link can cost someone their password. Training used `class_weight='balanced'` to punish missed phishing harder, and the threshold is adjustable. That bias is deliberate and disclosed on the site's Methodology tab.

The decision process, step by step

1. Start simple: LR baseline + RF benchmark, same data, same features
2. Measure: 5-fold CV + a 21,471-URL test set both models never saw
3. Check honesty: CV F1 $\approx$ test F1 for both -> no fluke, no leakage
4. Ask the product question: where must the model RUN?
5. Browser + exact parity + explanations -> LR ships, RF benchmarks

If you remember one thing:

Model choice = accuracy x deployability x explainability. RF is the most accurate, LR is the most deployable and explainable - so PhishGuard ships LR to your browser and keeps RF as the proof the features work.

Numbers cheat-sheet

Model	F1	ROC-AUC	Speed (per URL)
RandomForest	0.888	0.954	CLI / benchmark
LogisticRegression	0.804	0.885	website (in-browser)

Both trained on 107,355 real URLs, 80/20 stratified split, 5-fold cross-validation. Verdict threshold 0.5.
Regenerate anytime: `python -m src.train (in training/)`.